

Informe CI/CD

Diagnóstico y corrección de fallas reales en el pipeline de integración continua
Equipo ACC — Aplicaciones Distribuidas (ISR-701)

Carlos José Carpio Mendoza Cristhian Daniel Pacheco Cárdenas
Robinson Rodrigo Cando Moreno Jeremy Alexis Álvarez Párraga

Equipo ACC
Facultad de Ciencias de la Computación
Universidad Técnica Estatal de Quevedo

Agosto 2026

Resumen

Este informe documenta la depuración real del job `integration` del pipeline de CI/CD (7 jobs), que fallaba de forma consistente en GitHub Actions pese a funcionar sin problema en las máquinas de desarrollo del equipo. Se describen, en el orden en que se descubrieron, tres causas raíz independientes – todas confirmadas con evidencia real de logs, nunca supuestas – incluyendo los intentos previos que *no* resolvieron el problema, la corrección definitiva de cada una, y la verificación de punta a punta contra el stack real, hasta dejar los 7 jobs en verde sobre el mismo commit. Se documentan también los bugs secundarios encontrados en el propio proceso de verificar cada corrección, y las lecciones que deja el patrón común a los tres problemas.

Índice

1. Contexto	3
1.1. Metodología de diagnóstico	3
2. Problema 1: candado circular en el healthcheck de CockroachDB	4
2.1. Síntoma inicial	4
2.2. Primer intento: aumentar el presupuesto de tiempo (insuficiente)	4
2.3. Diagnóstico con logs reales	4
2.4. Causa raíz real	4
2.5. Solución definitiva	5
2.6. Verificación	5
3. Problema 2: datos de demo faltantes en el contrato Pact	5
3.1. Síntoma	5
3.2. Investigación	6
3.3. Solución	6
3.4. Bug adicional encontrado durante la verificación: UPSERT vs. índice único	6
3.5. Verificación	7

4. Problema 3: Playwright solo instalaba Chromium	7
4.1. Síntoma	7
4.2. Causa raíz	7
4.3. Solución	7
4.4. Verificación	8
5. Resultado	8
6. Amenazas a la validez	8
7. Conclusiones	9

1. Contexto

El equipo ACC mantiene un pipeline de integración continua en `.github/workflows/ci-cd.yml`, compuesto por 7 jobs encadenados mediante `needs`: para que cada etapa solo se ejecute si la anterior pasó:

Tabla 1: Los 7 jobs del pipeline

Job	Qué hace
<code>lint</code>	Análisis estático de todos los subproyectos (backend, web, móvil).
<code>test-backend</code>	Pruebas unitarias e integración de los microservicios Java, Node.js y Python.
<code>test-web</code>	Pruebas unitarias y de contrato (Pact) de <code>apps/web</code> .
<code>test-mobile</code>	Pruebas unitarias de <code>apps/mobile</code> (Kotlin/Jetpack Compose).
<code>build-images</code>	Matriz de 7 destinos: construye y publica en GHCR la imagen Docker de cada microservicio y de <code>apps/web</code> . Solo publica en push, no en pull request.
<code>build-mobile-apk</code>	Compila el APK de depuración de <code>apps/mobile</code> y lo publica como artefacto del pipeline.
<code>integration</code>	Levanta el stack completo con <code>docker compose up -d -build</code> y corre contra él: verificación del proveedor del contrato Pact y la suite completa de Playwright E2E en 3 navegadores.

Los primeros 6 jobs pasaban de forma consistente. El séptimo, `integration`, fallaba en *todas* las corridas sobre GitHub Actions, mientras que el mismo `docker compose up -d -build` que ese job ejecuta funcionaba sin problema en las máquinas de desarrollo del equipo. Esa asimetría – funciona en local, falla siempre en el runner – fue la pista central: el problema no era una falla puntual ni un error de sintaxis, sino una diferencia estructural entre ambos entornos que el equipo tuvo que identificar con precisión, no adivinar.

1.1. Metodología de diagnóstico

El primer obstáculo fue de *visibilidad*, no de código: la API pública de GitHub (`checks-runs/.../annotations`) solo devuelve mensajes genéricos del tipo "Process completed with exit code 1", sin el log real del comando que falló, y descargar el log completo de un run sin autenticación devuelve 403 `Must have admin rights`. Con solo esa API, cada intento de arreglo era en el fondo una conjetura educada sobre qué pudo haber fallado.

El equipo instaló y autenticó la CLI oficial de GitHub (`gh`), que sí da acceso al log completo de cada paso individual del job (`gh run view <id>-log-failed`). Este cambio de herramienta fue lo que permitió pasar de “adivinar la causa y esperar a ver si el siguiente run pasa” a “leer el error real, confirmarlo, y recién entonces corregirlo” – el patrón que se repite en los tres problemas de este informe. En Windows, la instalación de `gh` vía `winget` presentó además una complicación menor: el PATH actualizado no era visible para sesiones de terminal ya abiertas, lo que obligó a invocar el binario por su ruta absoluta hasta abrir una sesión nueva.

2. Problema 1: candado circular en el healthcheck de CockroachDB

2.1. Síntoma inicial

El paso `docker compose up -d --build` fallaba con `dependency failed to start: container roach1 is unhealthy`, después de aproximadamente 70 segundos de espera.

2.2. Primer intento: aumentar el presupuesto de tiempo (insuficiente)

La hipótesis inicial, razonable dado que un runner compartido de GitHub Actions tiene menos CPU disponible que una máquina de desarrollo dedicada, fue que CockroachDB simplemente necesitaba más tiempo para formar el clúster bajo contención de recursos. Se subió el presupuesto del healthcheck:

```
1 healthcheck:
2   test: ["CMD", "curl", "-f", "http://localhost:8080/health?ready=1"]
3   interval: 10s
4   timeout: 5s
5   retries: 15      # antes: 5
6   start_period: 60s # antes: 20s
```

Listing 1: Primer intento: mas tiempo, no resolvió el problema

El resultado del siguiente run real: la falla *seguía ocurriendo*, ahora después de aproximadamente 3 minutos y 21 segundos en vez de 70 segundos. Aumentar el tiempo no cambió el resultado, solo lo retrasó – la señal clara de que la causa no era de tiempo, sino estructural.

2.3. Diagnóstico con logs reales

Como ningún log de CockroachDB se había capturado nunca (el propio `docker compose up` fallaba y abortaba antes de llegar a un paso posterior que sí volcaba logs), se modificó el paso del workflow para volcar los logs de los 3 nodos ante cualquier falla:

```
1 docker compose up -d --build || {
2   echo "FALLO 'docker compose up' -- logs de los 3 nodos de CockroachDB:"
3   docker compose logs roach1 roach2 roach3
4   exit 1
5 }
```

Listing 2: Paso de diagnostico agregado al workflow

El siguiente run reveló, por fin, el mensaje real de CockroachDB en los 3 nodos: “The server appears to be unable to contact the other nodes in the cluster... Node will now attempt to join a running cluster, or wait for cockroach init”.

2.4. Causa raíz real

El healthcheck de los 3 nodos usaba `/health?ready=1` – un endpoint de *readiness* que en CockroachDB solo responde OK una vez que el nodo ya pertenece a un clúster **inicializado**. El servicio `db-init`, que es quien ejecuta `cockroach init` para inicializar ese clúster, tenía como condición `depends_on: roach1/2/3: service_healthy` – es decir, esperaba a que los nodos estuvieran “listos” antes de poder inicializarlos.

Candado circular real: nadie queda listo porque nadie los inicializa, y nadie los inicializa porque nadie queda listo. Ningún valor de `retries/start_period`, por grande que fuera, podía resolver esto – el candado nunca se abre por sí solo, sin importar cuánto tiempo se le dé.

En las máquinas de desarrollo del equipo esto nunca se manifestó porque los volúmenes de Docker ya traían un clúster inicializado de sesiones de trabajo anteriores: los nodos arrancaban, encontraban su directorio de datos ya inicializado, y pasaban el healthcheck de inmediato. El runner de GitHub Actions, en cambio, arranca siempre desde un volumen completamente vacío, así que el candado se activa sin excepción, cien por ciento de las veces.

2.5. Solución definitiva

- Los 3 healthchecks se cambiaron a `/health` (sin `?ready=1`) – verifica solo que el proceso está vivo (*liveness*), no que el clúster ya esté inicializado. Esto desbloquea a `db-init` para que pueda arrancar y ejecutar `cockroach init`.
- Se agregó un bucle de reintentos alrededor de `cockroach init` (hasta 30 intentos, 2 s entre cada uno) y una segunda espera activa a que el clúster acepte conexiones SQL antes de correr los scripts de esquema, para cerrar una carrera residual pequeña entre “el nodo terminó de inicializarse” y “el nodo ya acepta SQL real”.

```
1 for i in $(seq 1 30); do
2   out=$(cockroach init --insecure --host=roach1 2>&1) && break
3   echo "$out" | grep -qi 'already been initialized' && break
4   echo "db-init: intento $i fallo, reintentando en 2s: $out"
5   sleep 2
6 done &&
7 for i in $(seq 1 30); do
8   cockroach sql --insecure --host=roach1 -e 'SELECT 1;' >/dev/null 2>&1 && break
9   echo "db-init: el cluster aun no acepta SQL (intento $i), reintentando en 2s"
10  sleep 2
11 done &&
12 cockroach sql --insecure --host=roach1 -f /scripts/init_db.sql && ...
```

Listing 3: Extracto del entrypoint corregido de db-init

2.6. Verificación

Antes de subir el cambio se reprodujo el escenario de un entorno limpio en local, con una red y volúmenes de Docker completamente nuevos (sin reutilizar los datos ya inicializados del equipo), confirmando de punta a punta que el clúster se inicializaba correctamente sin intervención manual. La siguiente corrida real en GitHub Actions confirmó `roach1/roach2/roach3: Healthy` y el job `integration` avanzó más allá de este punto por primera vez en todo el proceso de depuración.

3. Problema 2: datos de demo faltantes en el contrato Pact

3.1. Síntoma

Resuelto el problema anterior, el job avanzó considerablemente más lejos – hasta la verificación del lado proveedor del contrato Pact, penúltimo paso del job – y ahí falló con:

```
1 1) Verifying a pact between soporte-web and ticket-service - una peticion GET
2   de la lista de tickets, sin filtros, con un JWT valido de ADMIN has a
3   matching body
4
5   1.1) body: $.data Variant at index 0 ({"ticketId":"c8e57689-...",
```

```
6 "status":"RESUELTO", ...}) was not found in the actual list
```

Listing 4: Fallo real de la verificación del contrato

El ticket que el contrato esperaba encontrar en la respuesta real de `ticket-service` simplemente no existía en la base de datos.

3.2. Investigación

El método `@State` de la prueba (`TicketServiceProviderPactTest`) era, hasta ese momento, un no-op a propósito, con un comentario explícito que decía que el stack “ya tiene datos de demo con tickets visibles para ADMIN”, citando un script, `resultados/rebalance_demo_tickets.sql`, como la fuente de esos datos. Se buscó ese archivo en el repositorio completo, incluyendo todo el historial de commits, y se confirmó que **nunca se llegó a versionar en git** – no existe ni existió jamás en ningún commit. Solo había existido, de forma manual y nunca documentada como tal, en el volumen de Docker de la persona que lo escribió originalmente en su propia máquina.

Es exactamente el mismo patrón de fondo que el Problema 1: la máquina de desarrollo “funcionaba” solo porque conservaba un estado acumulado que un entorno realmente limpio nunca podría reproducir por su cuenta.

3.3. Solución

Se reescribió el método `@State` para que inserte, por SQL directo, exactamente los dos tickets que el archivo de contrato (`pacts/soporte-web-ticket-service.json`) necesita – es precisamente para eso que existe el mecanismo `@State` de Pact: que la propia prueba garantice su precondition de forma explícita y reproducible, en vez de depender de un dato externo invisible que nadie versionó.

```
1 @State("el rol ADMIN tiene al menos un ticket visible")
2 void elRolAdminTieneAlMenosUnTicketVisible() throws Exception {
3     try (Connection conn = DriverManager.getConnection(
4         "jdbc:postgresql://localhost:26257/ticket_db?sslmode=disable",
5         "root", "")) {
6         // ... inserta por SQL directo exactamente los tickets que el
7         // archivo de contrato espera encontrar.
8     }
9 }
```

Listing 5: Extracto del `@State` corregido

3.4. Bug adicional encontrado durante la verificación: UPSERT vs. índice único

Al probar esta corrección contra el stack real (no simulada) se encontró un segundo bug, más pequeño pero real: el primer intento usó `UPSERT INTO` para insertar los tickets, y falló con “`duplicate key value violates unique constraint tickets_id_key`”.

La causa: `UPSERT` en CockroachDB solo resuelve conflictos contra la *primary key* compuesta de la tabla (`created_at`, `id`); la tabla `tickets` tiene además un índice único *aparte* sobre `id` en solitario (`tickets_id_key`, agregado para que las búsquedas por `id` no tengan que escanear las 4 particiones). Como esos tickets de demo ya existían de pruebas manuales anteriores en el mismo stack, con un valor distinto de `created_at`, el `UPSERT` no los reconoció como el mismo registro y chocó contra ese índice en vez de actualizar la fila.

```
1 -- Antes (fallaba si el ticket ya existia con otro created_at):
2 UPSERT INTO tickets (created_at, id, ...) VALUES (?, ?, ...)
```

```

3
4 -- Despues (idempotente sin importar el estado previo de la tabla):
5 DELETE FROM tickets WHERE id = ?;
6 INSERT INTO tickets (created_at, id, ...) VALUES (?, ?, ...);

```

Listing 6: Reemplazo de UPSERT por DELETE + INSERT

3.5. Verificación

Se corrió la prueba dos veces seguidas contra el stack real con Maven real (no una ejecución simulada): la primera corrida expuso el bug del UPSERT en vivo, con el error real de CockroachDB; tras aplicar el fix, ambas corridas pasaron 2/2 de forma consistente, confirmando la idempotencia real del mecanismo sin importar si los tickets ya existían de una corrida anterior.

4. Problema 3: Playwright solo instalaba Chromium

4.1. Síntoma

Con los dos problemas anteriores resueltos, el job llegó por primera vez hasta las pruebas E2E de Playwright. Los 3 casos de Chromium pasaron:

```

1 ok 1 [chromium] un ADMIN inicia sesion y ve la consola con tickets reales
2 ok 2 [chromium] abrir un ticket muestra su detalle completo
3 ok 3 [chromium] credenciales invalidas muestran un error real del backend
4
5 1) [firefox] un ADMIN inicia sesion...
6     Error: browserType.launch: Executable doesn't exist at
7     /home/runner/.cache/ms-playwright/firefox-1538/firefox/firefox

```

Listing 7: Chromium paso, Firefox y WebKit fallaron de inmediato

Firefox y WebKit fallaban de inmediato, sin siquiera llegar a intentar abrir una página – el navegador simplemente no estaba instalado en el runner.

4.2. Causa raíz

`playwright.config.ts` define 3 proyectos (`chromium`, `firefox`, `webkit` – requisito de “Compatibilidad” de la rúbrica de Entrega 4: la web debe funcionar en Chrome, Firefox y Safari, usando WebKit como el mismo motor de renderizado que usa Safari). El paso de instalación del workflow, sin embargo, decía:

```

1 npx playwright install --with-deps chromium

```

Listing 8: Paso de instalacion original, restringido a un solo navegador

El argumento `chromium` restringía la instalación a un solo navegador de los tres que la configuración realmente usa – una desconexión simple entre lo que el `.yaml` del pipeline instalaba y lo que el propio `playwright.config.ts` del proyecto exigía.

4.3. Solución

```

1 npx playwright install --with-deps

```

Listing 9: Correccion: sin argumento, instala los navegadores que la config usa

4.4. Verificación

Se instalaron los 3 navegadores en local (confirmando la presencia real de los binarios de `firefox` y `webkit` en la caché de Playwright) y se corrieron las 9 pruebas E2E completas contra el stack real levantado, confirmando que ningún caso volvía a fallar con el error de ejecutable faltante, en ningún navegador, antes de subir el cambio.

5. Resultado

Con los tres cambios integrados en commits sucesivos sobre la misma rama, el pipeline completo – los 7 jobs, incluidos los 7 destinos de la matriz de `build-images` y `build-mobile-apk` – quedó en verde sobre un mismo commit, cumpliendo el requisito de la rúbrica de que los 7 jobs pasen juntos en el último commit de la entrega.

Tabla 2: Resumen de las tres causas raíz encontradas y corregidas

Problema	Causa raíz	Corrección	Intento previo fallido
Healthcheck de CockroachDB	Candado circular: <code>?ready=1</code> exige clúster ya inicializado, pero quien inicializa espera a ese mismo healthcheck	<code>/health</code> (liveness) + reintentos de <code>cockroach init</code> + espera activa a SQL	Aumentar <code>retries/start_period</code> – solo retrasó la falla
Contrato Pact	Datos de demo asumidos como ya existentes, en un script nunca versionado en git	El propio <code>@State</code> inserta los datos por SQL (<code>DELETE+INSERT</code>)	UPSERT – chocó contra un índice único aparte
Playwright E2E	Paso de instalación restringido a un solo navegador (<code>chromium</code>)	Instalar los 3 navegadores que la configuración define	–

6. Amenazas a la validez

- **Un solo entorno de runner observado.** Todas las corridas se hicieron sobre los runners `ubuntu-latest` compartidos de GitHub Actions; no se probó el pipeline en otro proveedor de CI, así que no se puede afirmar que estas sean las únicas causas posibles de falla en un entorno “limpio” en general, solo que lo fueron en el entorno real usado por el equipo.
- **El primer intento del Problema 1 no se descartó por análisis, sino por observación.** Aumentar el presupuesto de tiempo era una hipótesis razonable a priori; se supo que era insuficiente solo después de ver el resultado del siguiente run real, no por haber anticipado el candado circular de antemano. Esto es consistente con la metodología descrita (diagnosticar con logs reales antes de dar un fix por bueno), pero vale dejarlo explícito.
- **La verificación de idempotencia del Problema 2 se hizo con 2 corridas, no más.** Es suficiente para confirmar que el bug del UPSERT se corrigió, pero no constituye una prueba exhaustiva de que no exista alguna otra condición de carrera no observada en ese punto.

7. Conclusiones

Los tres problemas comparten un mismo patrón de fondo: algo que *parecía* funcionar porque la máquina de desarrollo del equipo conservaba un estado acumulado – un volumen ya inicializado, datos de demo cargados a mano, o simplemente un navegador ya instalado de una sesión anterior – que un entorno realmente limpio, como el de un runner de GitHub Actions, nunca reproduce por su cuenta. Ninguno de los tres era un problema de “CI/CD mal configurado” en abstracto: cada uno tenía una causa raíz concreta y verificable con el log real del paso que fallaba, y cada corrección – incluidos los dos bugs secundarios encontrados en el propio proceso de verificar las correcciones principales – se probó de punta a punta contra el stack real, nunca simulada, antes de darla por buena.

Una lección práctica adicional: el primer intento de arreglar el Problema 1 (aumentar tiempos de espera) fue una hipótesis razonable que resultó incorrecta, y solo se supo que era incorrecta observando el resultado real del siguiente run, no por haberla anticipado de antemano. Aumentar un timeout es, en general, un cambio de bajo riesgo y fácil de justificar – pero cuando la causa real es estructural (un candado circular, en este caso), ningún valor de tiempo, por grande que sea, lo resuelve. La señal de que había que seguir buscando no fue que el fix “no funcionara” en abstracto, sino que el mismo síntoma reapareciera después de más tiempo en vez de desaparecer.